



### **Tarea 3: Polars**

**Curso:** Computación paralela y distribuida

**Profesor:** Johansell Villalobos Cubillo

**Estudiante:** Guissell Betancur Oviedo

# Descripción del Dataset

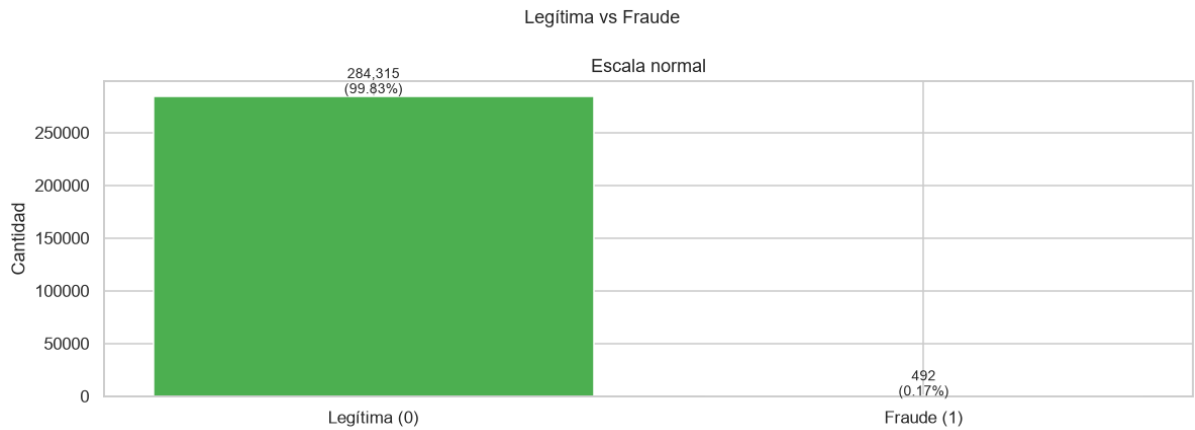
El dataset utilizado es el Credit Card Fraud Detection, disponible en Kaggle. Contiene transacciones con tarjeta de crédito en Europa para septiembre de 2013 registradas durante dos días consecutivos

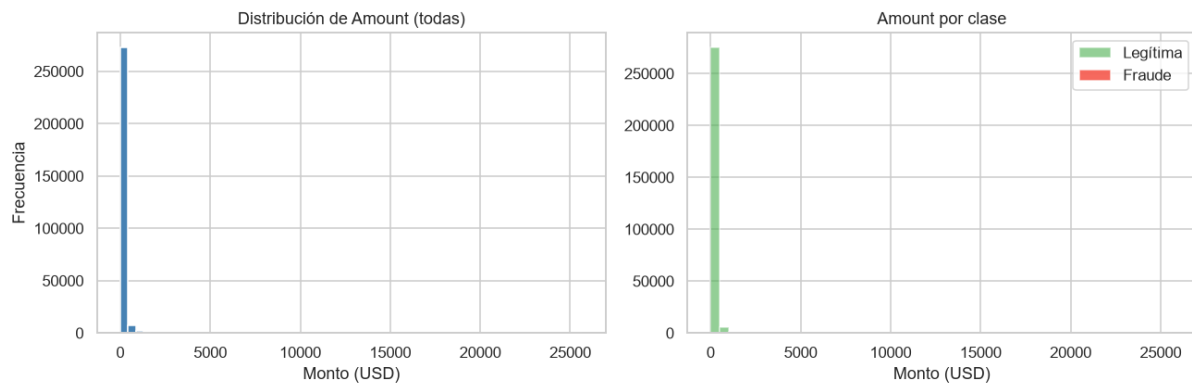
| Campo                  | Detalle                                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Nombre                 | Credit Card Fraud Detection                                                                                                   |
| Fuente                 | <a href="https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud">https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud</a> |
| Total de registros     | 284,807 transacciones                                                                                                         |
| Número de columnas     | 31 (Time, V1-V28, Amount, Class)                                                                                              |
| Variable objetivo      | Class: 0 = legítima, 1 = fraude                                                                                               |
| Distribución de clases | 284,315 legítimas (99.83%) / 492 fraudes (0.17%)                                                                              |
| Valores faltantes      | Ninguno                                                                                                                       |

Las variables V1 a V28 son resultado de un PCA realizado para proteger a los usuarios. Las únicas variables sin transformar son:

- Time: segundos desde la primera transacción
- Amount: monto de la transacción
- Class: variable objetivo

El dataset está desbalanceado, solo el 0.17% de las transacciones son fraudulentas. Por lo que se van a realizar técnicas que ayuden con el desbalance





## Pipeline implementado

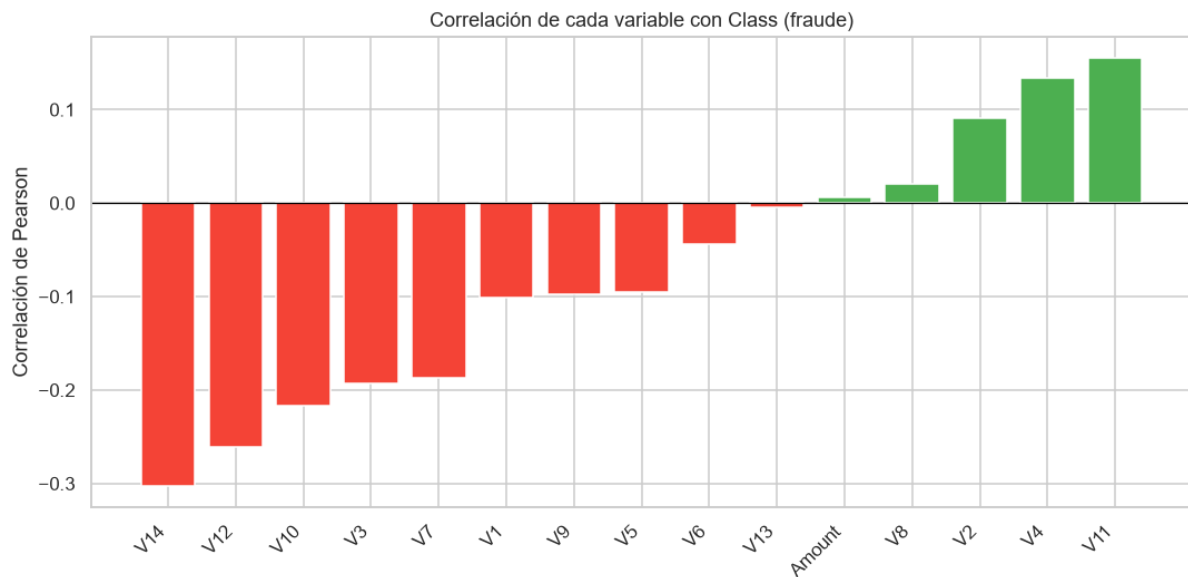
### Análisis exploratorio

Se utilizó Polars para el análisis exploratorio inicial:

- Estadísticas descriptivas de las variables Time, Amount y Class
- Verificación de valores faltantes: el dataset no contiene nulos
- Distribución de la variable objetivo
- Distribución del monto de transacciones por clase
- Tasa de fraude por hora del día mediante `group_by` en Polars
- Análisis de correlación de Pearson de las variables V1-V14 y Amount con Class

Algo a notar del EDA es que las horas 2 y 4 de la madrugada tienen la mayor tasa de fraude 1.71% y 1.04% respectivamente, puede que sucedan cuando hay menor monitoreo





## Ingeniería de características

Se usó Polars en los siguientes pasos:

- Filtrado: eliminación de filas sin etiqueta y montos negativos. (El dataset ya estaba limpio)

```
Filas antes : 284,807
Filas después: 284,807
Eliminadas : 0
```

- Nuevas variables: se creó Hora (extracción de la hora del día a partir de Time) y Amount\_log (logaritmo base 10 del monto, para reducir el sesgo de valores extremos).

```
shape: (5, 4)
┌ Time ─┬ Hora ─┬ Amount ─┬ Amount_log ─┐
│ --- │ --- │ --- │ --- │
│ f64 │ i64 │ f64 │ f64 │
├───┬───┬───┬───┤
│ 0.0 │ 0 │ 149.62 │ 2.177883 │
│ 0.0 │ 0 │ 2.69 │ 0.567026 │
│ 1.0 │ 0 │ 378.66 │ 2.579395 │
│ 1.0 │ 0 │ 123.5 │ 2.095169 │
│ 2.0 │ 0 │ 69.99 │ 1.851197 │
```

- group\_by + join: se calculó la tasa histórica de fraude por hora del día y se agregó al dataframe principal mediante un join. Esta nueva variable fraude\_tasa\_hora enriquece el conjunto de features.

```
shape: (5, 3)
┌ Hora ─┬ fraude_tasa_hora ─┬ Class ─┐
│ --- │ --- │ --- │
│ i64 │ f64 │ f64 │
├───┬───┬───┤
│ 0 │ 0.00078 │ 0.0 │
│ 0 │ 0.00078 │ 0.0 │
│ 0 │ 0.00078 │ 0.0 │
│ 0 │ 0.00078 │ 0.0 │
│ 0 │ 0.00078 │ 0.0 │
```

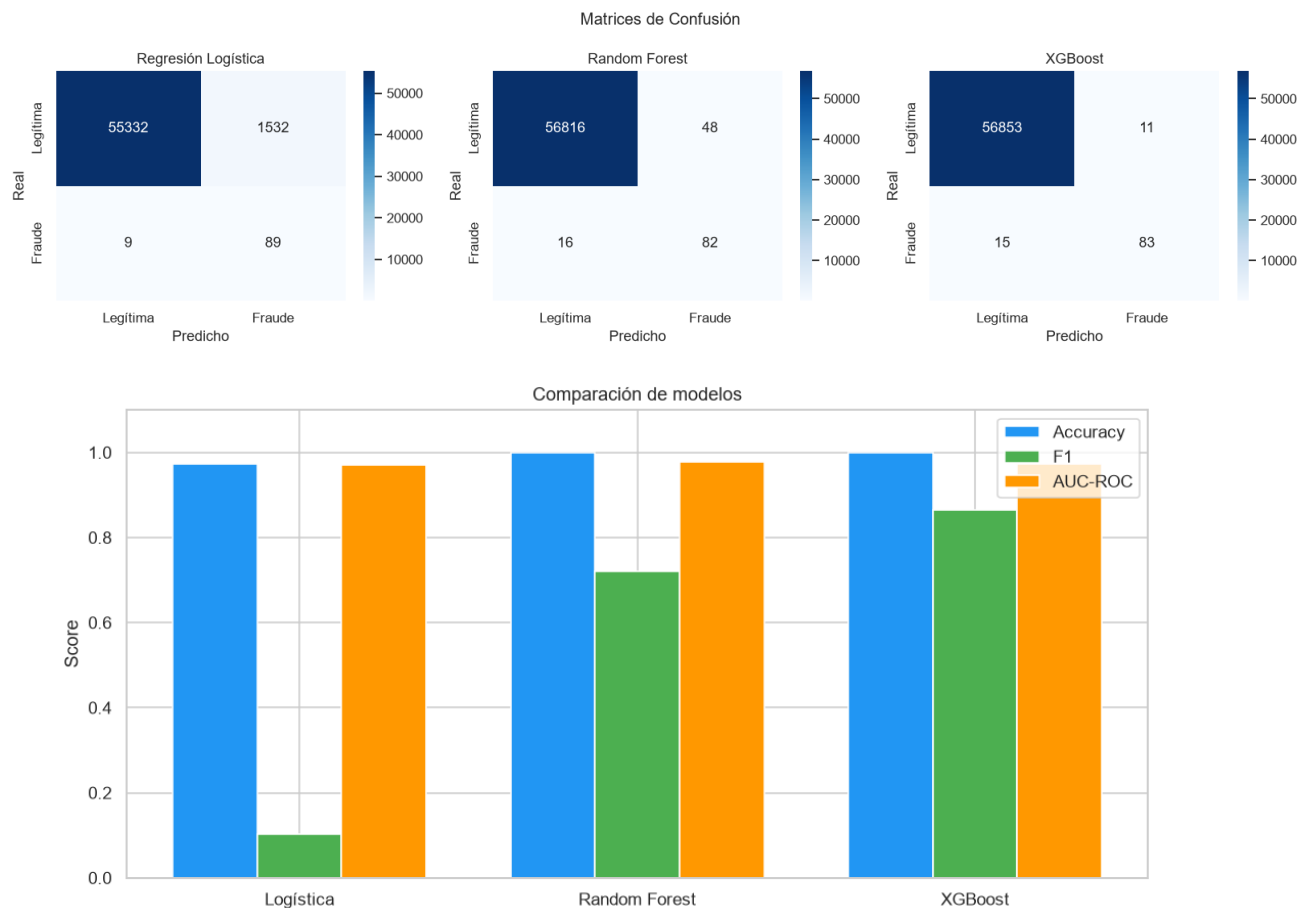
- Manejo de nulos: se iba a implementar imputación con mediana en Amount, Time y con cero (V1-V28) pero no fue necesaria dado que el dataset no tiene nulos.

```
Nulos antes : 0
Nulos después: 0
```

## Machine learning

Se entrenaron tres modelos de clasificación usando las 31 features más las 3 creadas. El split fue 80% entrenamiento y 20% prueba con estratificación, también por el desbalance se utilizó `class_weight='balanced'` en Regresión Logística y Random Forest y `scale_pos_weight` en XGBoost.

Resultados:



| Modelo              | Precision | Recall | Accuracy | F1     | AUC-ROC | Tiempo (s) |
|---------------------|-----------|--------|----------|--------|---------|------------|
| Regresión Logística | 0.05      | 0.91   | 0.9729   | 0.1035 | 0.9707  | 1.22       |
| Random Forest       | 0.63      | 0.84   | 0.9989   | 0.7193 | 0.9770  | 25.93      |
| XGBoost             | 0.88      | 0.85   | 0.9995   | 0.8646 | 0.9727  | 5.57       |

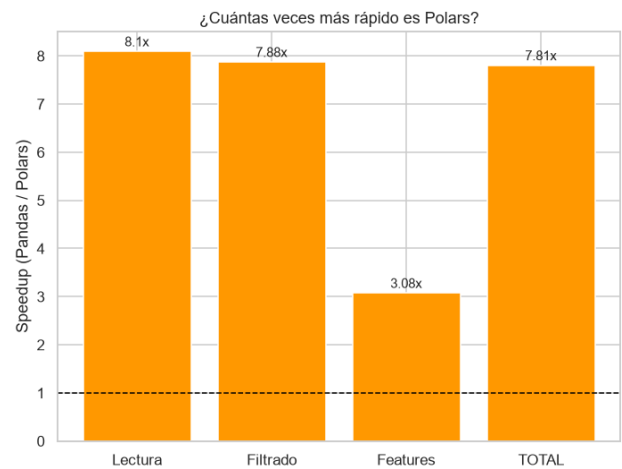
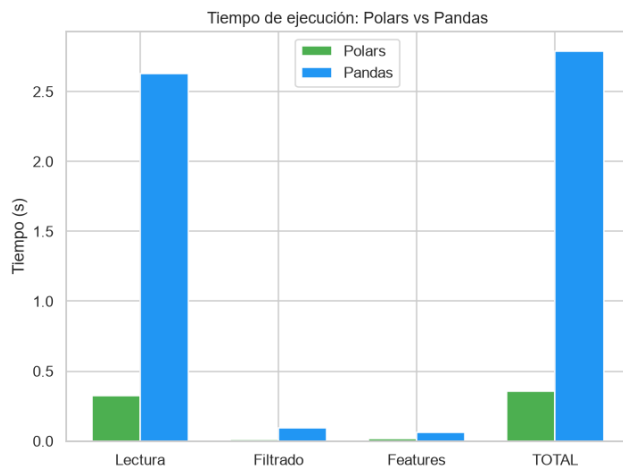
- **Regresión Logística:** Aunque obtuvo un AUC-ROC de 0.97 y detecta el 91% de los fraudes (recall alto), la su precisión para fraude es de 0.05, significa que genera demasiadas falsas alarmas: de cada 100 alertas de fraude, solo 5 son fraudes reales. El F1 de 0.10 refleja este problema.

- **Random Forest:** Mejoró mucho el F1 a 0.72, con una precisión de 0.63 pero el tiempo de entrenamiento fue de 25.93 segundos siendo el más alto en comparación.
- **XGBoost - Mejor Modelo:** XGBoost obtuvo el mejor rendimiento general: F1 de 0.86, precisión de 0.88 (de cada 100 alertas, 88 son fraudes reales) y recall de 0.85, fue el más eficiente porque tardó solo 5.57 segundos en entrenar.

## Benchmark: Polar vs Pandas

Se implementó el mismo pipeline de preprocesamiento en ambas librerías y se midieron los tiempos de cada etapa

| Etapas       | Polars (s) | Pandas (s) | Speedup |
|--------------|------------|------------|---------|
| Lectura CSV  | 0.3248     | 2.6298     | 8.10x   |
| Filtrado     | 0.0123     | 0.0972     | 7.88x   |
| Feature Eng. | 0.0202     | 0.0621     | 3.08x   |
| TOTAL        | 0.3573     | 2.7891     | 7.81x   |



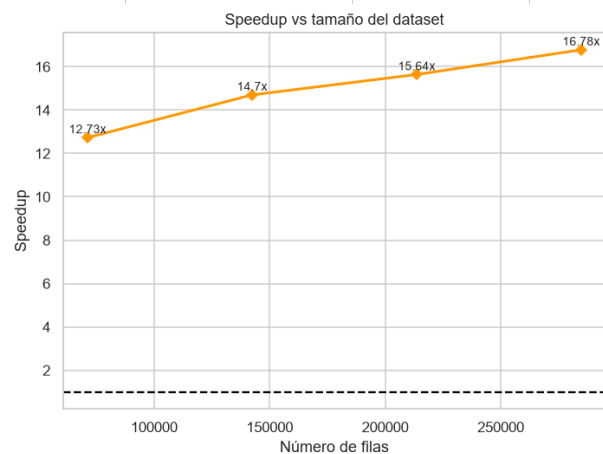
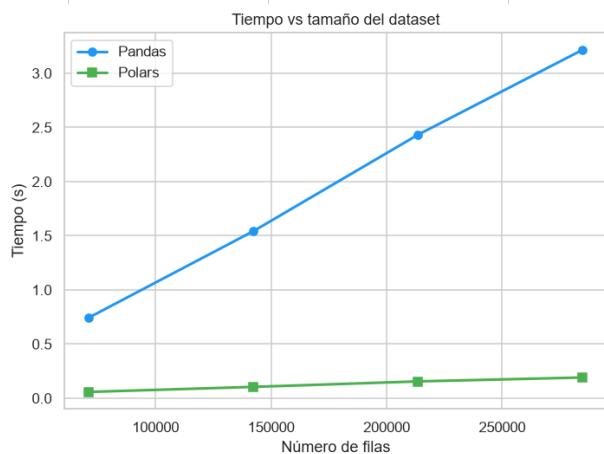
Polars demostró ser más rápido. La mayor diferencia se fue en la lectura del CSV y el filtrado, pudo ayudar el paralelismo automático de Polars. La ingeniería de características mostró el menor speedup probablemente porque las 2 librerías son eficientes en operaciones de agregación.

## Experimentos

### Escalabilidad con tamaño de datos

Se construyeron subconjuntos del 25%, 50%, 75% y 100% del dataset original para medir cómo escala el rendimiento de cada librería

| Fracción | Filas   | Pandas (s) | Polars (s) | Speedup |
|----------|---------|------------|------------|---------|
| 25%      | 71,201  | 0.7452     | 0.0585     | 12.73x  |
| 50%      | 142,403 | 1.5432     | 0.1050     | 14.70x  |
| 75%      | 213,605 | 2.4327     | 0.1555     | 15.64x  |
| 100%     | 284,807 | 3.2164     | 0.1917     | 16.78x  |

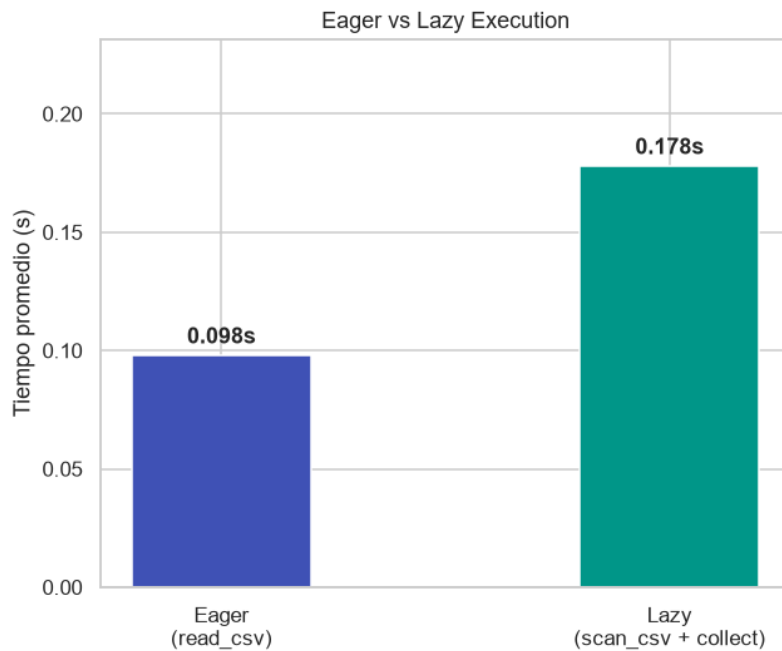


El speedup de Polars aumenta con el tamaño del dataset: de 12.73x con el 25% hasta 16.78x con el 100% de los datos. Polars parece aprovechar mejor el paralelismo a medida que hay más datos para procesar, lo que lo hace adecuado para grandes volúmenes de información

### Escalabilidad con tamaño de datos

Se compararon los modos de lectura Eager (`pl.read_csv`) y Lazy (`pl.scan_csv + .collect()`) sobre 3 repeticiones

| Modo                                     | Tiempo promedio (s) | Descripción                               |
|------------------------------------------|---------------------|-------------------------------------------|
| Eager ( <code>read_csv</code> )          | 0.098s              | Lee todo el CSV en memoria de una vez     |
| Lazy ( <code>scan_csv + collect</code> ) | 0.178s              | Construye un plan optimizado y lo ejecuta |



En este experimento, Eager resultó más rápido que Lazy para la operación de lectura simple. Según lo analizado puede ser porque tiene un plan de consulta que no se justifica cuando solo se lee el archivo sin operaciones adicionales y le va mejor cuando se hacen muchas operaciones.

## Análisis de resultados

### 1. ¿Qué ventajas observó al utilizar Polars?

Polars fue 7.8x más rápido en el pipeline completo. Es sencillo de entender la sintaxis, si se vio una buen performance.

### 2. ¿Qué operaciones obtuvieron el mayor speedup?

La lectura del CSV (8.1x) y el filtrado (7.88x) les fue mejor, probalmente por el paralelismo automático de Polars en operaciones de I/O y selección de datos.

### 3. ¿En cuáles operaciones la diferencia fue pequeña?

group\_by y join obtuvo el menor speedup (3.08x). Parece que ambas son rápidas cuando hacen operaciones de agregación.

### 4. ¿Qué beneficios aporta Lazy Execution?

Permite optimizar el plan completo antes de ejecutarlo, me recordó un poquito a spark, al apreper es útil en pipelines con muchas operaciones encadenadas. Para operaciones sencillas como fue leer un CSV Eager puede ser más rápido porque evita el overhead de construir el plan.

### 5. ¿Qué limitaciones encontró en Polars?

Requiere pyarrow para convertir a Pandas, lo que puede causar problemas de compatibilidad. Tiene menor compatibilidad directa con scikit-learn. Su documentación y comunidad son más pequeñas que las de Pandas.

### 6. ¿Qué ventajas mantiene Pandas?

Puede ser mayor compatibilidad con librerías de ML y visualización, más recursos de aprendizaje disponibles, más intuitivo para usuarios nuevos y con mayor soporte de la comunidad.

### 7. ¿La aceleración justifica migrar un proyecto existente?

Para pipelines con más de 100K filas por la mejoría podría ser. El speedup reduce bastante los tiempos de procesamiento. Para proyectos pequeños puede no ser necesario



**8. ¿Cómo afecta el tamaño del dataset al beneficio obtenido?**

El speedup creció con el tamaño del dataset: de 12.7x con el 25% de los datos hasta 16.8x con el 100%. Polars aprovecha mejor el paralelismo a mayor volumen de datos.

**9. ¿Qué modelo produjo el mejor desempeño predictivo?**

XGBoost obtuvo el mejor F1 (0.86) y precisión (0.88) para detectar fraudes, siendo además más rápido que Random Forest (5.57s vs 25.93s).

**10. ¿Qué recomendaciones daría para proyectos futuros?**

Usar Polars para ETL y preprocesamiento de datos grandes. Mantener Pandas para exploración rápida y compatibilidad con ML. Para datos desbalanceados, siempre usar `class_weight` o `scale_pos_weight`. Aprovechar Lazy Execution cuando el pipeline tenga muchas operaciones encadenadas.

## Conclusiones

Polars demostró ser una herramienta más eficiente que Pandas para el procesamiento de datos, con un speedup promedio de 7.8x en el pipeline completo y hasta 16.78x al escalar al dataset completo. La mayor ventaja se observó en lectura y filtrado gracias al paralelismo automático.

En cuanto a los modelos, XGBoost fue el más efectivo para manejar el desbalance de clases, obteniendo un F1 de 0.86 con solo 5.57 segundos de entrenamiento. La Lazy Execution resultó más útil en pipelines con múltiples operaciones encadenadas que en lecturas simples.

Para proyectos futuros se recomienda usar Polars para ETL con grandes volúmenes de datos, mantener Pandas para compatibilidad con librerías de ML, y siempre aplicar técnicas de manejo de desbalance como `class_weight` o `scale_pos_weight` en problemas de clasificación similares.